Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Davide Fissore
Université Côte d'Azur, Inria, France

Enrico Tassi
Université Côte d'Azur, Inria, France

Abstract

We mechanize two operational semantics for Prolog with cut and prove them equivalent, in the Rocq prover. The first semantics closely models the Elpi's implementation, it is based on a stack of choice points and is well suited for studying optimizations employed by Prolog abstract machines. The second semantics is instead based on and-or trees, in which the branches pruned by the cut operator are made explicit.

We use the tree-based semantics (1) to prove properties about the effect of the cut operator in the evaluation of choice points, since the rules from classical logic do not apply, and (2) to reason about the determinacy analysis introduced in Elpi version 3.0, which statically identifies computations that produce at most one result.

2012 ACM Subject Classification Theory of computation → Operational semantics

Keywords and phrases Semantics, Prolog, Elpi, Determinacy Analysis, Rocq, Coq

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Funding *Davide Fissore*: This work has been supported by the French government, through the France 2030 investment plan managed by the Agence Nationale de la Recherche, as part of the "UCA DS4H" project, reference ANR-17-EURE-0004.

1 Introduction

Elpi is a dialect of λProlog (see [16, 17, 7, 13]) used as an extension language for the Rocq prover (formerly the Coq proof assistant). Elpi has become an important infrastructure component: several projects and libraries depend on it [14, 3, 4, 21, 8, 9]. Examples include the Hierarchy-Builder library-structuring tool [5] and Derive [19, 20, 12], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, Elpi is equipped with a static analysis for determinacy [10] to help users control backtracking. Rocq users are familiar with functional programming, but not necessarily with logic programming; in particular, uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checker allows the user to assert which predicates should be considered deterministic and then checks that these predicates behave like functions, i.e., they commit to their first solution and leave no *choice points* (places where backtracking could resume).

This paper reports our first steps towards a mechanization, in the Rocq prover, of the determinacy analysis from [10]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for Prolog with cut. The first is a stack-based semantics that closely models Elpi's implementation and is similar to the semantics mechanized by Pusch in Isabelle/HOL [18] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations used by standard Prolog abstract machines [22, 1], but it makes reasoning about the scope of cut difficult. To address this limitation we introduce a tree-based semantics in which the

```
func f int -> int. % f is a deterministic predicate. The two rules
f 1 2.             % have non-overlapping heads, i.e. 1 != 2
f 2 3.
pred r int -> int. % r is a non-deterministic predicate. The heads
r 0 0.             % overlap: the query for 0 has three applicable
r 0 1.             % rules.
r 0 2 :- !.
func g int -> int. % g is deterministic: if the first rule succeeds, the
g A C :- r A B, f B C, !.  % second one is cut away, as well as the choice
g D F :- f D E, f E F.     % points left by the call to r.
```

Figure 1 Running example of a Elpi program

branches pruned by cut are explicit and we prove the two semantics equivalent. Using the tree-based semantics, we then prove some properties about the impact of evaluating the cut in disjunctive querys. For example, unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut-away by q_1.

Finally, we use the formal semantics to prove the correctness of a static determinacy analysis [10]. Intuitively, nondeterminism arises when a computation can be completed by applying two different rules. We exclude this situation in two ways. First, we check for overlapping rule heads: if two heads do not overlap in the inputs they accept, then no query can unify with both. Second, we account for the presence of cut in rule premises: once a rule whose premises contain a cut succeeds, all remaining rules are discarded and all choice points allocated by premises preceeding the cut are discarded. We show that if every rule defining a predicate passes this analysis, then any call to that predicate leaves no choice points.

impreciso: possiamo avere chiamate a relazioni nel body

2 Preliminaries: syntax and informal semantics of cut

Both semantics share the same notion of program, written $\mathbb{P}$. Figure 1 is our running example. A program consists of a list of rules `R` together with a signature environment `sigT`. Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, and whether the predicate is deterministic.

A rule consists of a head term `Tm` and a list of premises. Each premise is an `Atom`, i.e. either the cut operator or a predicate call. Terms `Tm` include predicate symbols, data, and variables. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ-calculus style (no parentheses).

The set of variables `V` is countable. We represent substitutions Σ as finitely supported maps from variables to terms, using the finmap library [15]. The empty substitution is written ε while the composition of two substitutions σ_1 and σ_2 with disjoint supports as $\sigma_1 + \sigma_2$.

The backchaining operation applies all rules to a query term; it is central to both semantics.

```
Inductive Tm := ...
  | Pred of P | Var of V.
Inductive sig := NonDet | Det.
Definition sigT :=
  P -> sig * nat * nat.
```

Figure 2 Syntax of Elpi programs

```
Inductive A := cut | call : Tm → A.
Record R := { head : Tm; premises : list A }.
Record ℙ := { rules : list R; sig : sigT }.
Definition Σ := {fmap V → Tm}.
```

Figure 3 Syntax of Elpi programs

```
Definition bc : ℙ → 𝓕ᵥ → Tm → Σ → 𝓕ᵥ * list (Σ * list A) :=
```

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the set of variable names used so far ($\mathcal{F}_\nu$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

In our mechanization we abstract the unifier and its properties, namely

```
Record Unif := {
  unify : Tm → Tm → Σ → option Σ;
  matching : Tm → Tm → Σ → option Σ;
}.
```

TODO: state axioms and explain matching; cite Dale's unification results. In this paper we assume the reader is familiar with first-order unification.

2.1 The cut operator

The cut operator in Elpi implements the *hard cut* (see, e.g., the documentation of SWI-Prolog [**?**]). Whenever the backchaining function `bc` returns a list of alternatives, the first one is explored immediately, while the others are suspended as *choice points*. Within a given alternative, premises are processed from left to right. When a cut is encountered, it discards (i) all choice points created by `bc` for the current call and (ii) all choice points created while executing the premises preceding the cut.

For example, consider the program P in Figure 1 and the query `g 0 R`. Both rules for `g` apply to this query. Backchaining therefore returns the following two alternatives:

$$[(\sigma_1, [\texttt{r A B, f B C, !}]),\ (\sigma_2, [\texttt{f D E, f E F}])]$$

with $\sigma_1 := \{\texttt{A} \mapsto 0, \texttt{B} \mapsto \texttt{C}\}$ and $\sigma_2 := \{\texttt{D} \mapsto 0, \texttt{F} \mapsto \texttt{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_\nu$ any further.

Execution continues with the first premise of the first alternative, namely `r A B` under σ_1, i.e. `r 0 C`. The remaining alternatives are stored as choice points. Backchaining `r 0 C` returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [!])]$ where $\sigma_3 := \sigma_1 + \{\texttt{B} \mapsto 0\}$; $\sigma_4 := \sigma_1 + \{\texttt{B} \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{\texttt{B} \mapsto 2\}$.

Since all rules for `r` have empty premises, we chose the first success, save the others as choice points, and continue with `f B C` under σ_3, that is, `f 0 C`. This time, backchaining fails (no rule for `f` matches input 0). We therefore backtrack to the most recently introduced choice point and retry `f B C` under σ_4, i.e. `f 1 C`. This succeeds with $\sigma_6 := \sigma_4 + \{\texttt{R} \mapsto 2\}$.

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for `r` and one from the second rule for `g` (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for `g` was selected. (In contrast, a cut does not discard choice points created earlier; in this example, there are none.)

The final solution is: $\{\texttt{A} \mapsto 0, \texttt{B} \mapsto 1, \texttt{C} \mapsto \texttt{R}, \texttt{R} \mapsto 2\}$.

In Sections XX and YY we provide fully mechanized operational semantics for Prolog with cut. They differ in how they represent the ongoing computation, in particular how alternatives are stored and how they are pruned by cut.

```
Inductive tree :=
  | KO | OK                              (* failure and success *)
  | Todo of Atom                         (* unexplored branch *)
  | Or   of option tree & Σ & tree       (* A ∨ B_σ *)
  | And  of tree & list Atom & tree.     (* A ∧_{B_0} B *)

Inductive tag := Expanded | CutBrothers | Failed | Success.
Definition step : ℙ → F_ν → Σ → tree → (F_ν * tag * tree) :=
Definition next_alt : 𝔹 → tree → option tree :=
Inductive runT : ℙ →  F_ν → Σ → tree → option (Σ * option tree) → 𝔹 → F_ν → Prop :=
```

Figure 4 Signatures of the tree-based semantics

2.1.0.1 Notational conventions

In the following section we note $\mathbb{B}$ for the bolean type, and $\top$ and $\bot$ for `true` and `false`. The option instances are noted $\bullet t$ for *Some* t and $\square$ for *None*. Following the Mathematical Components library, on which we depend, we use `x.1` and `x.2` to denote the first and second projection applied to the pair `x`.

spiegare notation for list

3 And-Or Tree semantics

TODO: nested option, output Fv also inside option

An And-Or tree is a stratified, variable-width tree. It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, but all subqueries in the corresponding And-layer.

The tree is built incrementally. It therefore contains a `Todo` node, which represents an unexplored branch (an `Atom`). When the atom is a `call`, we use the backchaining procedure from Section 2 to compute the two layers to attach to the tree: alternatives form the Or-layer, and the premises of each applicable rule form the corresponding And-layer. This expansion step is performed by the `step` procedure described in Section 3.2.

The Rocq data type for And-Or trees is given in Section 3. In addition to `Todo`, it provides two distinguished nodes, *KO* and *OK*, which represent failed and completed branches (intuitively, an empty disjunction and an empty conjunction, respectively).

The *Or* node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A. The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ, which becomes the current substitution when backtracking to B.

why option and not KO?

The *And* node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to be solved after A. We call B_0 the *reset point* for B: it is used to restore the initial state of B when backtracking occurs within A. An example is shown in Section 3.3.

A graphical representation of a tree is shown in Figure 6a. For compactness, we depict *And* and *Or* nodes as n-ary (rather than binary), with children associating to the right. The *KO* node acts as the terminating element of an *Or* list, while *OK* is the terminating element of an *And* list.

```
Fixpoint next σ t : Σ * tree :=
  match t with
  | Todo _ | KO | OK ⇒ (σ, t)
  | Or □ σ' B ⇒ next σ' B
  | Or •A _ _ ⇒ next σ A
  | And A _ B ⇒
    let (σ', pA) := next σ A in
    if pA == OK then next σ' B
    else (σ', pA)
  end.
```

```
Definition next_subst σ t := (next σ t).1
Definition next_tree t := (next ε t).2.

Definition success t := next_tree t == OK.
Definition failed t := next_tree t == KO.
Definition incomplete t :=
  if next_tree t is Todo _ then ⊤ else ⊥.
```

Figure 5 *next* and derived functions

3.1 The tree-based semantics

As anticipated, the tree is constructed incrementally by two recursive functions, *step* and *next_alt*. Both traverse the tree depth-first, left-to-right. Since all functions must terminate in Rocq, we define their transitive closure as the inductive relation *runT*.

Intuitively, *runT* iterates calls to *step* as long as they succeed (tag `Success`). Upon failure (tag `Failed`), it searches for the next alternative and continues from there, if one exists. We detail *step*, *next_alt*, and *runT* in Sections 3.2–3.4.

To support incremental construction, the semantics uses helper functions based on *next*. Given a tree, *next* returns the branch on which the execution should focus, following a depth-first, left-to-right traversal strategy (see Section 3.1).

In all trees in Figure 6, a black arrow marks the result of *next_tree*.

3.2 The *step* procedure

The *step* procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a *tag*, and an updated tree.

The set of variables is assumed to contain all variables occurring in the tree. It is passed to backchain so that rules can be refreshed correctly.

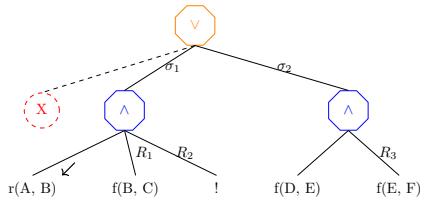
The *tag* (see Section 3) indicates which kind of step was performed. `Expanded` indicates the expansion of a predicate call via backchain. `Failure` and `Success` are returned for trees that satisfy, respectively, the *failed* and *success* tests. Finally, `CutBrothers` indicates a superficial cut: *step* was called on an Or-layer and the And-layer immediately below it contains a cut.

The *step* procedure evaluates atoms. In particular, it leaves the tree unchanged when *next_tree* is false.

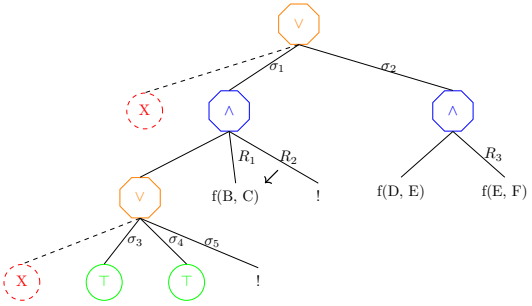
```
Lemma succ_step_iff: ∀ p v σ t, success t ↔ step p v σ t = (v, Success, t).   (1)
Lemma fail_step_iff: ∀ p v σ t, failed t ↔ step p v σ t = (v, Failed, t).     (2)
```

step expands `Todo` nodes by calling backchain, which returns a list l of alternatives. If l is empty, then the current call is replaced by *KO*. Otherwise, it is replaced by a right-skewed tree of *Or* nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf is *OK*; otherwise, it is a right-skewed tree of *And* nodes.

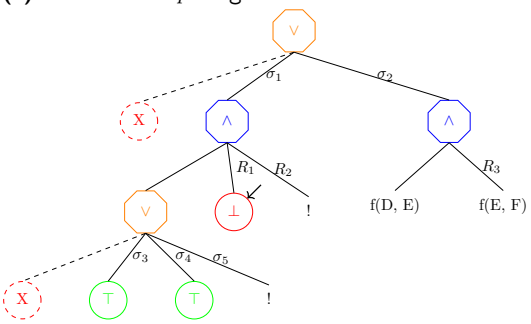
[1] The substitutions $\sigma_1 \ldots \sigma_6$ are from Section 2.1. R_1, R_2, and R_3 are reset points and correspond respectively to the lists of atoms [`f Y Z, !`], [`!`], and [`f Y Z`].



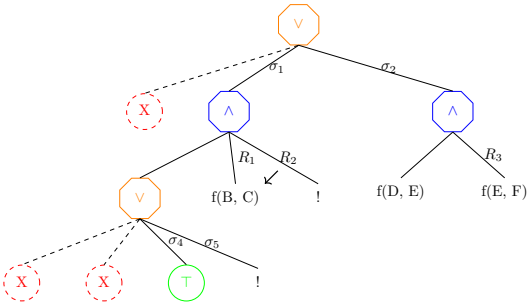
(a) Tree after *step* on `g 0 R`



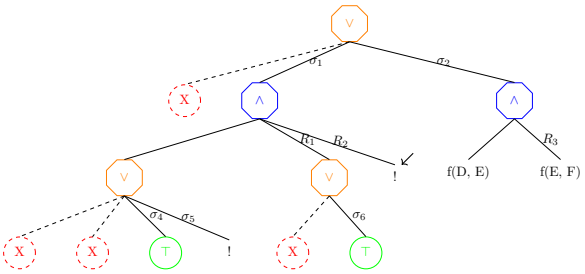
(b) *step* on Figure 6a



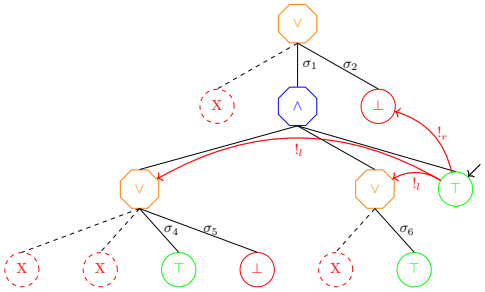
(c) *step* on Figure 6b



(d) *next_alt* on Figure 6c



(e) *step* on Figure 6d



(f) *step* on Figure 6e

Figure 6 Some tree representations[1]

Figure 6a depicts the tree corresponding to the running example (Section 2.1) as it undergoes calls to *step*. We run query `q 0 R` against the program P in **??**. Let c_0, c_1, and c_2 be the children of the root. By construction, c_0 is the $\square$ tree, while c_1 and c_2 correspond respectively to the premises of rules r_1 and r_2 in P. We need the initial $\square$ subtree so that we can attach a substitution to c_1. In particular, c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in Section 2.1. Reset points are marked with the R symbol: $R_1 \coloneqq$ [`f Y Z, !`], $R_2 \coloneqq$ [`!`], and $R_3 \coloneqq$ `f Y Z`. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of *step* triggering backchaining appear in Figures 6b, 6c, and 6e.

sottolineare le sostituzioni

3.2.1 The interpretation of a cut

The step corresponding to a cut performs two actions: (i) the cut node is replaced by *OK*; and (ii) the subtrees in the scope of *Cut* are pruned. More precisely, (ii) prunes the left siblings of the *Cut* node (in the same And-layer, denoted $!_l$) and prunes the right uncles of *Cut* (in the one-level-up Or-layer, denoted $!_r$).

An example of the impact of cut is shown in Figure 6f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. We note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by *KO*. This amounts to discarding the third rule of predicate `r` (see Section 2.1).

3.3 The *next_alt* procedure

When backchain returns an empty list, *step* produces the tag `Failed` and the computation must backtrack. The *next_alt* procedure is responsible for producing a new tree from which the computation can continue.

In Figure 6c we encounter a failure because no rule applies to the atom `f X Y` under substitution σ_3, which maps `Y` to `0`. The *next_alt* procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the *OK* node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the *KO* node with the information stored in the reset point R_1. This restores the call `f Y Z`, which can then be reconsidered under substitution σ_4.

Furthermore, *next_alt* serves another important purpose. Its behavior depends on the boolean flag it receives as input: next_alt $\bot$ A recursively removes all failures (if any) from A, whereas next_alt $\top$ A removes the first success in A. For example, the tree in Figure 6f contains a successful path that maps `R` to `2`.

Some interesting properties of *next_alt* are shown below; they illustrate how *next_alt* complements *step* with respect to failed, success, and *path_atom* trees.

```
Lemma path_atom_next_alt_id: ∀ b t, incomplete t → next_alt b t = •t.   (3)
Lemma next_alt_failedF: ∀ b t t', next_alt b t = •t' → failed t' = ⊥.   (4)
Lemma next_altFN_fail: ∀ t, next_alt ⊥ t = □ → failed t.                (5)
```

3.4 The *runT* inductive

The inductive relation *runT* relates four inputs to three outputs. The inputs are a program, a set of variables, an initial substitution σ, and a tree t. The outputs are (i) an optional result r of the form $\bullet(\sigma', t')$, (ii) a boolean b, and (iii) an updated set of variable names.

The main output is r: if $r = \square$, interpretation fails; otherwise σ' is the computed solution substitution and t' represents the remaining, not-yet-explored alternatives. The boolean b records whether a superficial cut was evaluated during execution, and the updated variable set tracks freshly generated names.

runT has four cases, depicted as inference rules in Figure 7. (StopT) If *next_tree* t is a success, then the substitution σ' is extracted from t (starting from σ) and the input tree is cleaned of its successful path. (StepT) If *next_tree* t is an atom, then *step* is invoked to evaluate t, and *runT* is called recursively on the updated tree. (BackT) If *next_tree* t is a failure, then *next_alt* is called to clear the failed path; if the resulting cleaned tree exists, *runT* is called recursively on it. Finally, (FailT) accounts for trees with no solutions.

These rules are deterministic:

```
Lemma runT_det: ∀ p v0 σ t r r' b b' v v',
  runT p v0 σ t r b v → runT p v0 σ t r' b' v' → r' = r ∧ v' = v ∧ b' = b.
```
(6)

The proof is by induction on the first *runT* derivation and is immediate, because the rules are selected based on *next_tree* t. In particular, the hypotheses *success* t, path_atom t, and *failed* t are mutually exclusive. Moreover, by Lemma 6, the hypothesis of FailT implies *failed* t; hence FailT is exclusive with StopT and StepT, and it is also exclusive with BackT since they impose different requirements on the output of *next_alt*.

$$\frac{\text{success } t \qquad \text{get\_subst } \sigma\ t = \sigma' \qquad \text{next\_alt } \top\ t = t'}{\text{runT } v\ \sigma\ t\ \bullet(\sigma', t')\ \bot\ v}\ \textsc{StopT}$$

$$\frac{\begin{array}{cc}\text{path\_atom } t & b' = \text{is\_cb tg} \mid\mid b \\ \text{step p } v\ \sigma\ t = (v', \text{tg}, t') & \text{runT } v'\ \sigma\ t'\ r\ b\ v''\end{array}}{\text{runT } v\ \sigma\ t\ r\ b'\ v''}\ \textsc{StepT}$$

$$\frac{\text{failed } t \qquad \text{next\_alt } \bot\ t = \bullet t' \qquad \text{runT } v\ \sigma\ t'\ r\ \text{n}\ v'}{\text{runT } v\ \sigma\ t\ r\ \text{n}\ v'}\ \textsc{BackT}$$

$$\frac{\text{next\_alt } \bot\ t = \square}{\text{runT } v\ \sigma\ t\ \square\ \bot\ v}\ \textsc{FailT}$$

Figure 7 Rule system for *runT*

3.5 Properties of *runT*

Working with *runT* allows to prove some interesting properties about the behavior of the cut operator wrt disjunction, i.e. the *Or* node. We have proven several properties that are avalable at *this* link. Below we show two of these lemmas.

```
Lemma runSST_or: ∀ p v v' σ σ' l l' σ1 r,
  runT p v σ l •(σ', •l') ⊤ v' →
    runT p v σ (Or •l σ1 r) •(σ', •(Or •l' σ1 KO)) ⊥ v'.
```
(7)

```
Lemma runNT_or: ∀ p v v' σ t σ1 t',
  runT p v σ t □ ⊤ v' →
    runT p v σ (Or •t σ1 t') □ ⊥ v'.
```
(8)

```
Lemma run_orSST p v v' σ σ' σ1 l l' r r':
  runT p v σ (Or •l σ1 r) •(σ', •(Or •l' σ1 r')) ⊥ v' →
  ∃ b, runT p v σ l •(σ', •l') b v' ∧ r' = if b then KO else r.
```
(9)

```
Definition stepE v t σ a g :=
  let (v', r) := bc p v t σ in
  (v', save_alts a g r).
```

$$\frac{}{\text{runE } v\ ((\sigma, \varnothing) :: a)\ \bullet(\sigma, a)}\ \textsc{StopE} \qquad \frac{\text{runE } v\ ((\sigma, g) :: ca)\ \text{r}}{\text{runE } v\ ((\sigma, (\text{cut}, ca) :: g) :: \_)\ \text{r}}\ \textsc{CutE}$$

$$\frac{\text{stepE } v\ t\ \sigma\ a\ g = (v_1, a') \qquad \text{runE } v_1\ (a' +\!\!+ a)\ \text{r}}{\text{runE } v\ ((\sigma, (\text{call } t, \_) :: g) :: a)\ \text{r}}\ \textsc{CalleE} \qquad \frac{}{\text{runE } \_\ \varnothing\ \square}\ \textsc{FailE}$$

Figure 8 Rule system for *runE*

In Lemma 7, we evaluate the tree l. This evaluation produces a success with signature σ' and a new tree l'. During the evaluation, a superficial cut in l is traversed. This implies that the evaluation of $\bullet l \vee_{\sigma_1} r$ returns σ' as substitution, and the alternatives are $\bullet l' \vee_{\sigma_1}$ KO. The right-hand side of the *Or* node is replaced with *KO* by $!_r$.

In Lemma 8, we are in a similar scenario: t evaluates a superficial cut but, this time, produces a failure. This implies that the evaluation of $\bullet t \vee_{\sigma_1} t'$ also fails.

Finally, in Lemma 9, we evaluate $\bullet l \vee_{\sigma_1} r$. The evaluation succeeds, producing the result $\bullet(s_1, \bullet l' \vee_{\sigma_1} r')$. This means that l still has l' as unexplored alternatives. We deduce that the evaluation of l produces the substitution σ_1 with alternatives l'. However, we have no information about whether a superficial cut has been crossed in l. If this is the case, we know that r' is cut away by $!_r$; otherwise, r' has not been explored and is equal to r.

From these lemmas, it is clear how the order in which alternatives are evaluated changes the behavior of the program.

se riesco success (A or B) = success (B or A) if no sup cut in A and B

4 Stack semantics

We now introduce the Elpi semantics. The interpreter is close to that of the Elpi language and provides an operational semantics that is similar to the one proposed by Pusch in [18], which is in turn closely related to the semantics given by Debray and Mishra in [6, Section 4.3]. Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract Machine [22, 1].

We represent a state of the Elpi language thanks to the two mutual inductives shown below.

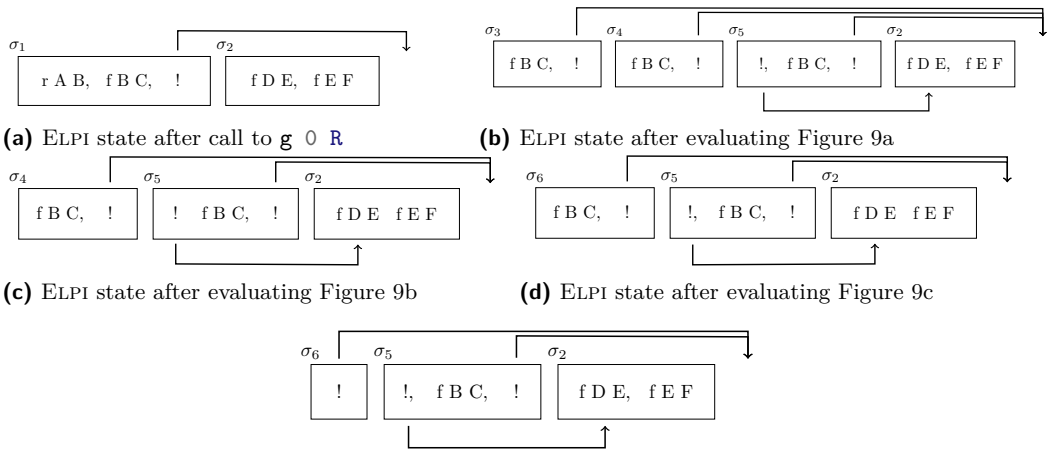
```
Inductive alts  := nilA | consA of (Σ * goals) & alts
with       goals := nilG | consG of (A * alts)  & goals .
```

An Elpi state is an enhanced two-dimensional list. The outermost list represents a list of alternatives in disjunction, each accompanied by the substitution that should be used for their interpretation. The innermost list is a list of atoms, that is, the list of goals in conjunction. These goals are decorated with a pointer to an alternative list, which is used to keep track of where to jump when a cut is interpreted. We call these special alternatives the *cut-to* alternatives.

The Elpi interpreter is called *runE* and has the following type.

```
Inductive runS : F_ν → alts → option (Σ * alts) → Prop :=        (1)
```

The inductive *runE* represents a routine taking as input a set of variables and a list of alternatives and returning an option. □ stands for interpretation with no solution,



(a) ELPI state after call to `g 0 R`

(b) ELPI state after evaluating Figure 9a

(c) ELPI state after evaluating Figure 9b

(d) ELPI state after evaluating Figure 9c

(e) ELPI state after evaluating Figure 9d

Figure 9 Example of ELPI execution

whereas $\bullet(\sigma, a)$ represent a successful interpretation with σ as solution and a as the list of non-yet-explored choice points.

Dire che non abbiamo le due info ritornate da runT

The rule system for *runE* is given in Figure 8 and is made by 4 cases. We distinguish two main cases: if the list of alternatives is empty, (FAILE) we have a failure: we stop the computation and return $\square$. If the list of alternatives is the list $(\sigma_0, h) :: a$, the interpreter evaluates the first choice point h under the substitution σ_0. If h is empty, (STOPE), then we have a success, i.e. we return the input σ_0 and leave a as the future choice points. Otherwise, h is the list $(t, ca) :: g$ where t is the first atom to evaluate, ca is its cut-alternative list and g is the list of future conjuncts. If t is a cut (CUTE), we need to keep evaluating g under σ_0 but we change the alternatives to ca. Finally, in the case of a call (CALLE), we backchain the call in the program. If rs is the list of rule premises returned by `bc`, then each list of premises if mapped to a list of goals (`goals`) have a as cut-alternative and the global list is translated to a list of alternative (`alts`).

Dire una parola su backtracking = tl if bc = nil add subst to each alternative in Figure 9 detail more?

Example of execution We propose an exemple of execution of an ELPI state in Figure 9. The arrows in the images represent the cut-alternative pointer of the cut. We have not put arrow goiing our from atom-calls since the interpreter ignores the cut-alternatives in this case (cf. CALLE).

Remark We want finally to point out how the lemma that we were able to prove in Section 3.5 are not easy to be expressed in the ELPI semantics. This is beacuse there is no sufficient structure in the ELPI state to understand what is the scope of the cut operator. For example, in an ELPI state made of these alternatives $a_0, a_1 \dots a_n$, we have no clear idea of what happens if the first alternative a_0 succeeds: there could be a cut in a_0 but, is this cut superficial? What is its impact wrt the alternatives $a_1 \dots a_n$? More then that, it is also possible that the state is hill-formed and the cut does not point to a suffix in the list of alternatives.

5 Equivalence of the two semantics

In the previous section we have introduces the two semantics, they are The equivalence between the two semantics is possible under two conditions: we need to work with "valid tree", i.e. all of those trees that can be generated from a call. Secondly, we need to translate

```
Fixpoint valid_tree A :=
  match A with
  | Todo _ | OK | KO ⇒ ⊤
  | Or □ _ B ⇒ valid_tree B
  | Or •A _ B ⇒ valid_tree A &&
        ((B == KO) || B.base_or B)
  | And A BO B ⇒ valid_tree A &&
      if success A then valid_tree B
      else B == big_and BO
  end.
```

(a) Valid tree

```
Fixpoint t2l t σ (cp: alts) : alts :=
  match t with
  | OK     ⇒ [(σ, ∅)]
  | KO     ⇒ ∅
  | TA a   ⇒ [(σ, [(a, ∅)])]
  ...
```

(b) Tree to list

Figure 10 Valid tree and Tree to list

trees into elpi states. In the next to subsection we propose the two functions *valid_state* and *t2l*.

5.1 Valid trees (*valid_state*)

The inductive tree allows one to generate a large number of trees, some of which are not valid, in the sense that they cannot be produced starting from a given query. The class of valid trees is characterized by the function shown in Figure 10a.

While all base cases of tree are considered valid, we need to analyze carefully the cases for the *Or* and *And* constructors.

For the *Or* constructor, we distinguish two cases depending on whether the lhs exists. If it does not exist, then the ths must be a valid tree. Otherwise, the lhs must itself be a valid tree, and the rhs is either the *KO* tree, since it may have been removed by the evaluation of a superficial cut in the lhs, or it has not yet been explored. In the latter case, it is a `base_or` tree, namely the right-skewed tree formed by a disjunction of conjunctions that is genereated after a succeful backchain to a call.

For the *And* constructor, the lhs is required to be a valid tree. The shape of the rhs depends on whether the lhs is a success. If the lhs is not successful, then the rhs has never been explored: the procedures *step* and *next_alt* modify the rhs only when the lhs succeeds. In this case, the lhs must be the right-skewed tree containing conjunctions of premises atoms, the reset point B_0 ensure what shape the rhs should have. If the lhs is a success tree, then the rhs can be modified by *step* and *next_alt*, therefore it must be a valid tree.

5.2 From trees to lists (*t2l*)

The translation of a tree to a en Elpi state is done thanks to the *t2l* function. In Figure 10b we give junst the implementation of the base cases, the full implementation is available at *this* link.

The function *t2l* takes the tree t_0 we want to translate, a subsitution σ_0 and a list of choice points *cp*. These choice points represent alternatives that appears at the same level of the current node, such as, for example, the right-siblings of a *Or* node. The *OK* node represent one succeful alternative, thereofre it is translated into a singleton list with the empty list of goals. The *KO* node represent a failure, i.e. it is mapped to the empty list of alternatives. Finally, atoms are mapped to one alternative containing a single goal with the empty cut-alternative list. They cannot point to *cp* since they are not right-uncle subtrees.

t	6a	6b	6c and 6d	6e	6f
$t2l\ t\ \varepsilon\ \varnothing$	9a	9b	9c	9d	9e

Table 1 Tree to list from Figure 6 and Figure 9

We have omitted the code for the translation of the *Or* and *And* constructor since it is quite complex, but, we give the intuition behind this translation by means of examples.

The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B. The alternatives of the rhs are valid choice points for the lhs and should be passed to the translation of A. The two list of alternatives can be concatenated and, each atom, should add *cp* as new cut-to alternatives: A and B are one level lower wrt *cp* and therefore the cut they have should point to *cp*.

The translation of $A \wedge_{B_0} B$ implies the translation of A. If the translation of A returns an empty list we return the empty list of alternatives without even touching B nor B_0. Otherwise the translation returns a list $(\sigma_0, g) :: a$. By the semantics of $runT$ the B node represent the continuation of the first alternatives in A, i.e. g, whereas B_0 is the continuation of goals for each alternative in a. While performing these translation, we need to carefully update the cut-atlernatives.

dirlo meglio all'inizio

Example of t2l In Table 1 we point out the result of calling *t2l* on the trees in Figure 6. The first raw contains the image reference of then trees and the second raw contains the reference of the images from the ELPI state in Figure 9. For the 3^{rd} column in the table, we see that *t2l* is not injective. There are different trees that maps to the same ELPI state. In fact, there are transitions in $runT$ that are not visible in $runE$.

5.3 Equivalence theorems

Thanks to the two functions *valid_state* and *t2l*, we are able to finally state the equivalence theorems between the semantics. The proof we are giving are on a slightly different definition of $runT$, since in the equivalence, we ignore the verification for the boolean and the variable set returned by $runT$, these piece of data are paricularly meant for the proof in Section 3.5.

The definition we use is the following:

```
Definition runT' p v σ t r := ∃ b v', runT p v σ t r b v'.
```
(2)

Before going to the equivalence proofs, we want to claim a key lemma allowing to ignore the silent transition in the $runT$ inductive.

```
Lemma next_altF_t2l: ∀ t t' b,
  valid_tree t → failed t → next_alt b t = •t' →
    ∀ l σ, t2l t σ l = t2l t' σ l.
```
(10)

dire di v

It is proved easly by induction on the tree t.

```
Lemma tree_to_elpi: ∀ p t σ r,
  let v := vars_tree t `|` vars_sigma σ in
  valid_tree t →
    runT' p v σ t r →
      let a := t2l t σ ∅  in
      let r' :=  if r is •(σ', t) then  •(σ', t2l (odflt KO t) σ ∅ )
                 else □ in
      runS p v a r'.
```
(11)

Proof sketch We proceed by induction on the execution of $runT$'. There are 4 cases to be treated. The first case comes from STOPT, it deals with successful trees, a successful tree must to start with an empty list and therefore we conclude with STOPE. The case for

StepT, need to treat two cases: *step* has evaluated either a cut or a call. Depending on this we should use CutE or CallE and then the induction hypothesis. The case for BackT is a silent case: it represents the non-injective behavior of *t2l*, but, thanks to Lemma 10 and the induction hypotheses the conclusion ist proved. The last case comes from the derivation FailT. It is easily proved using FailE and the fact that if *next_alt* returns □ when trying to earse failure in t, then *t2l* of t returns the empty list.

```
Lemma elpi_to_tree p v a r :
  runS p v a r →
  ∀ σ t, valid_tree t → t2l t σ ∅  = a →
  if r is •(σ', a') then
    ∃ t', runT' p v σ t •(σ', t') ∧ t2l (odflt KO t') σ ∅  = a'
  else runT' p v σ t □.
```
(12)

The proof of Lemma 12 is based on the idea explained in [2, Section 3.5.1]. An ideal statement for this lemma would be: given a function *l2t* transforming an Elpi state a to a tree t, we then would be able to compare the result of *runE* run with a and *runT*' run with t. The function it is difficult to implement since we have hard time finding the tree structure of a, and more than that, we should have a notation of Elpi valid state, but again, this is a hard task. We thereofre need to reuse the *t2l* function to relate a tree t with a. Figure 11 show the idea of the proof: we have a state t whose translation is a, we proceed by induction on the execution of *runE*. This gives us not only the output a' but that hypothesis that it exists a tree t' such that its translation to the Elpi state is a'. This proof strategy is indirectly providing a kind of *l2t* function by combining the execution of the *runE* and *t2l*.

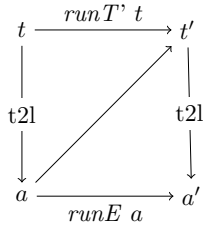


Figure 11

6 Case study: determinacy alanysis

An interesting application of the tree semantics is the determinacy checking that was added in version 3 of the Elpi langauge [11]. The checker is meant to receive the signature of the predicates declared by the user and then verify that the rules of a deterministic predicate will return at most one solution, we define this particular kind of predicate as functions. Thanks to this static verification, the user can better control unexpected backtracking at compile time: if an incoherence is rilevata then a fatal error explain why the current program may break determinacy. A function behaves deterministically if two main conditions are respected: *mutual exclusion* guarantees that at most one rule can be successfully used on a given query, whereas *rule checking* ensure that each rule does not creates choice points, e.g. by calling non-deterministic predicates.

For example, in **??**, we can declare the following signatures to the predicates. spiegare sintassi

`f : int →i int →o func` `g : int →i int →o func` `r : int →i int →o pred`

We see that `f` is a function since its rules are mutual exclusive and have empty body and `g` is a function since `r1` is cutting `r2` if `r1` succeeds due to the cut. Moreover, both rules ref verso programma

The signature of the program are stored in the its `sig` projection, and, as in [11] we need to extends the type of predicates so that we are able to distinguish between deterministic and non-deterministic propositions. For example, in **??**, we have ...

A program execution behaves deterministically if given a program, a substitution and a tree to the *runT* inductive, then either the execution fails, or, if is succeeds, then the tree of alternatives is □. Below we define `is_det`.

```
Definition is_det p σ v t :=
    ∀ r, runT' p v σ t r → if r is •(_, x) then x = □ else r = □.
```

use coq syntax instead of elpi?

The theorem we want to prove is the following.

```
Lemma det_check_call p σ t:
  let v := vars_tm t `|` vars_sigma σ in
  check_ℙ p → tm_is_det p.(sig) t → is_det p σ v (Todo (call t)).
```

With `check_ℙ` being the function checking the program wrt mutual exclusion and rule checking and `tm_is_det` being the function testing if the term head refers to a rigid constant with deterministically type. The proof of this lemma should be done on the derivation of the program execution. However, in this definition, the induction hypothesis is too weak, since in we would have to prove ... but we will not be able to show that the ... is deterministic.

To solve the problem, we need to work on "deterministic trees" (`det_tree`), show that if a tree respect this condition, and then prove the following theorem which is more generic then `det_check_call`.

```
Lemma det_check_tree σ v p t:
  vars_tree t `<=` v → vars_sigma σ `<=` v →
  check_ℙ p → det_tree p.(sig) t → is_det p σ v t.
```

Since `det_check_call` is an instance of `det_check_tree`, its proof is now trivial.

As an important remark, proving the same lemma with the stack semantics, it is difficult to correctly define the predicate `det_stack`, checking for the determinstc behavior of a generic stack, since, similar to Section 3.5, the lack of structure make this goal hard. We need therefore an intermediate data structure, like the tree to encompass this problem.

7 Related work

The input mode of Elpi, inspired by the XXXX of BProlog, has only an impact in the determinacy analysis that does not required to be preceeded by a mode analysis. Well moded program propagate the groundness from input arguments to output arguments. As a result if the initial query is ground then unification degenerates to matching (for input arguments). Considering a semantics where input arguments are match has the following impact on det analysis: axiom XXX does not require q to be ground.

The only mechanization of Prolog's semantics we could find is the one by Pusch in Isabelle/HOL [18] that is based on the model of Debray and Mishra [6, Sec. 4.3]. Their work start from that semantics and refines it by introducing some optimizations usually found in the Warren abstract machine [22, 1]. They do not use the semantics to prove properties on the execution of programs as we do in our use case, hence they do not face the difficulty described in seciton XXX that pushed us to develop an more explicit semantics (with respect to cut). In some way our work is complementary, showing he stack based semantics equivalent to more optmized one, we could consider implement in Elpi.

Another relevant work are the ones by King cite, but we could not find their code. Their semantics is denotational and cannot easily cover all the features our paper XX does, but would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by XXX and we thank Y.Bertot for insightful discussions on the topic. In XXX Bertot relates bla bla, witout never constructing a decompiler, exactly as we never build a list to tree function.

8 Conclusion

We do this. This is a first step for XXXX. We need to add substitutions. The missing part of the proof becomes related to abstract interpretation, since the det types for a lattice and one has to prove that the concrete runtime values variables take agree with the abstraction computed statically. This proof is ongoing and somehow orthogonal to the current one since the cut operator and the HO feature do not interfere with eachother in complex ways.

References

1. Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.
2. Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: https://inria.hal.science/inria-00073199v1.
3. Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.
4. Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.
5. Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPIcs*, pages 34:1–34:21, 2020. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.34, doi:10.4230/LIPIcs.FSCD.2020.34.
6. Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:10.1016/0743-1066(88)90007-6.
7. Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: https://inria.hal.science/hal-01176856v1, doi:10.1007/978-3-662-48899-7_32.
8. Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: https://inria.hal.science/hal-04467855.
9. Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.
10. Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In *Practical Aspects of Declarative Languages: 28th International Symposium, PADL 2026, Rennes, France, January 12–13, 2026, Proceedings*, pages 77–95, Berlin, Heidelberg, 2026. Springer-Verlag. doi:10.1007/978-3-032-15981-6_5.
11. Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.
12. Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. doi:10.1145/3573105.3575683.
13. Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. In *Mathematical Structures in Computer Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/S0960129518000427.

14 Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27, doi:10.4230/LIPIcs.ITP.2025.27.

15 math-comp. finmap — finite maps for mathcomp, 2026. URL: https://github.com/math-comp/finmap.

16 Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.

17 Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, 2012.

18 Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors, *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.

19 Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λProlog dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January 2018. URL: https://inria.hal.science/hal-01637063.

20 Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of *LIPIcs*, pages 29:1–29:18, September 2019. URL: https://inria.hal.science/hal-01897468, doi:10.4230/LIPIcs.CVIT.2016.23.

21 Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi. Master's thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.

22 David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309, SRI International, Artificial Intelligence Center, Computer Science and Technology Division, Menlo Park, CA, USA, October 1983. URL: https://www.sri.com/wp-content/uploads/2021/12/641.pdf.